

Intelligent Document Extraction, Validation & API Platform

Invoices · Balance Sheets · Profit & Loss · Cash Flow Statements

OCR + layout-aware extraction · financial reconciliation · REST API · dashboard

The problem

The same numbers arrive in inconsistent, often scanned, layouts

1

Validate

Type, integrity and page limit —
before OCR

2

Read

Native PDF text, or rasterise and
OCR

3

Extract

Fields and every table row,
grounded

4

Reconcile

Recompute the financial
relationships

5

Serve

Store, expose via API and
dashboard

Four document types in scope

- Invoice — header fields, line items, tax, cash and change
- Balance sheet — capital & liabilities and assets, per period
- Profit & loss — income, expenditure, profit and appropriations
- Cash flow — operating, investing, financing, opening/closing cash

Architecture

One FastAPI application serves the API and the dashboard

Intelligent Document Extraction, Validation & API Platform

Request path for POST /api/v1/documents/process

Browser (frontend/)

dashboard.html - document type + file upload, results table
document_result.html - fields, tables, validation, raw JSON
static/js - calls the REST API with fetch()

HTTPS

FastAPI application (backend/app/main.py)

POST /api/v1/documents/process (multipart upload)
GET /api/v1/documents (dashboard list)
GET /api/v1/documents/{name} (latest result)
GET /api/v1/health (liveness)
GET /docs (Swagger / OpenAPI)
AppError handlers -> {"error": {"code", "message"}}

Structured JSON response

document_name, document_type, processing_status
file_validation { type, readable, page_count }
extracted_data { field: value, page_number,
source_text } + line_items[]
validation { checks[], overall_status, issues[] }
processing_metadata { ocr_used, timings }

Cross-cutting

core/config.py - env-driven settings (no secrets in code)
core/logging.py - structured application logging
core/exceptions.py- typed errors -> HTTP status codes
core/database.py - engine + session lifecycle

Processing pipeline (services/)

orchestrated by document_service.py

1. document_validation_service.py

PDF / JPG / PNG by magic bytes, not the
declared type - integrity, page limit (<=3)

2. ocr_service.py

Native PDF text via pdflumber, else
rasterise + Tesseract; keeps word boxes

3. extraction_service.py + utils/layout.py

Rebuilds rows by y-position, assigns each
number to a period column by x-position

4. llm_extraction_service.py (optional)

Fills only fields left null; skipped
entirely when no API key is configured

5. financial_validation_service.py

Per-type checks with formula, operands,

External / system

Tesseract OCR (pytesseract)
Poppler pdftoppm (pdf2image)
Anthropic API - optional, key-gated

result

Why line-by-line OCR text is not enough

The insight that drove the design

Reading OCR text as lines

Total 743, 732, 155

A row filled in for only one year gives no clue which year it belongs to — and OCR often emits labels and numbers as separate blocks entirely.

Reading the page as a table

Total · x≈1540 → column 2 → 31-Mar-16

Word boxes are kept from both PDF text and OCR. Rows are rebuilt by vertical position; each number is assigned to a period column by its right edge.

Measured, not assumed

- Benchmarked all 30 dataset statements; the first extractor was over-fitted to one year
- Compared page-segmentation modes and DPI — 300 DPI measured worse than 200
- Extraction and validation had drifted to two different sets of field lookups; unified them
- A second colour OCR pass gained one check for twice the latency, so it was left out

Financial validation

Each check reports formula, operands, calculated vs reported, variance, status

Invoice

$qty \times unit\ price \approx line\ amount \cdot \sum lines \approx subtotal \cdot subtotal + tax - discount \approx total \cdot cash - total \approx change$

Balance sheet

$Total\ Capital\ \&\ Liabilities \approx Total\ Assets \cdot \sum component\ rows \approx each\ reported\ total$

Profit & loss

$Interest + Other\ Income \approx Total\ Income \cdot Income - Expenditure \approx Net\ Profit \cdot - Minority \approx Attributable$

Cash flow

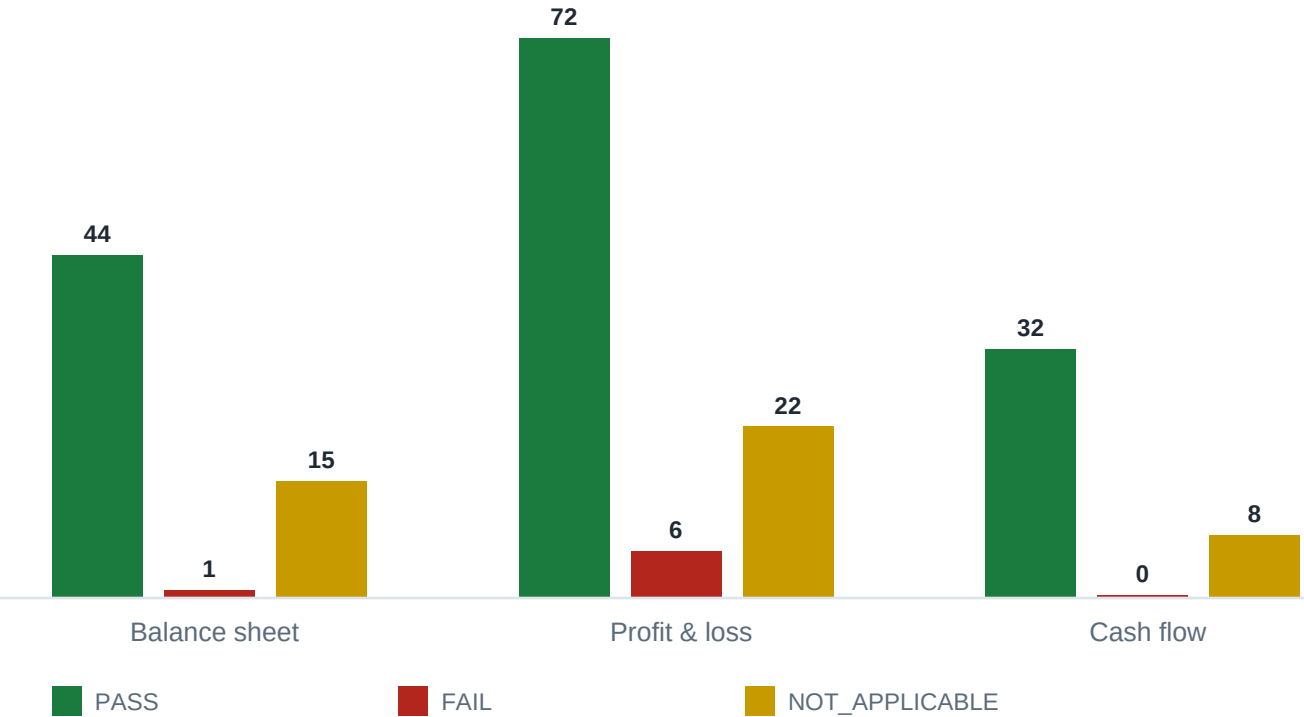
$Operating + Investing + Financing + FX \approx Net\ Increase \cdot Opening + Net\ Increase \approx Closing\ Cash$

Honest by design

- Missing operand → NOT_APPLICABLE, never a failure
- A partly-read section is not summed — an unread row would count as zero
- Tolerance: max(1.0, 1% of the larger value)
- Bracketed figures parse as negative
- Every check runs once per reporting period
- Missing values stay null — never guessed

Measured results

All 30 statements in the provided dataset (2017–2026), 200 checks



148

checks reconcile exactly

7

flagged as discrepancies

45

not applicable — unreadable scans

Clean scans reconcile fully; the not-applicable results concentrate in the 2020–2022 scans where OCR cannot recover row labels.

API and dashboard

One deployment serves both; Swagger is generated from the code

POST	/api/v1/documents/process	upload and process
GET	/api/v1/documents	dashboard list
GET	/api/v1/documents/{name}	latest result
GET	/api/v1/health	health check
GET	/docs	Swagger / OpenAPI

Errors return one envelope

```
{ "error": { "code": "UNSUPPORTED_FILE_TYPE",  
            "message": "Only PDF / JPG / PNG..." } }
```

415 · 413 · 400 · 404 · 422 · 500 — never a stack trace

Intelligent Document Extraction & Validation API

Back to dashboard

X51005361895.jpg

Type: invoice · Status: PASS · Processed at: 9/10/2026, 5:10:40 PM · OCR used: yes

Extracted fields

VENDOR NAME

Not found

VENDOR ADDRESS

Not found

VENDOR TAX ID

Not found

CUSTOMER NAME

Not found

INVOICE NUMBER

1CRO576494 (p.1)

INVOICE DATE

25/01/2018 (p.1)

INVOICE TIME

Not found

CURRENCY

Not found

SUBTOTAL

Not found

TAX AMOUNT

0.51 (p.1)

TAX RATE PERCENT

6 (p.1)

TAX INCLUSIVE

true

DISCOUNT

Not found

TOTAL AMOUNT

9 (p.1)

TOTAL QUANTITY

Not found

CASH PAID

50 (p.1)

CHANGE

41 (p.1)

Financial validation

Overall status: PASS

Check	Formula	Calculated	Reported	Variance	Status
tax_inclusive_total_check	(total_amount - tax_amount) + tax_amount ≈ total_amount (tax shown as included in total)	9	9	0	PASS
cash_change_check	cash_paid - total_amount ≈ change	41	41	0	PASS

Raw JSON

Show / hide

Limitations and what I'd change

Known today

- OCR quality is the binding constraint — the 2020–2022 scans lose row labels entirely
- Statement parsing is tuned to this dataset's banking format
- Noisy receipts yield fewer line items
- Synchronous processing, ~2–6 s per scanned page
- No authentication — per case study scope

For production

- Queue the work and return a job id instead of holding the connection
- Commercial OCR, with the LLM pass standard rather than opt-in
- Golden-file regression suite over a labelled corpus
- Object storage for source documents; Alembic migrations
- Auth, rate limiting, metrics and alerting on the FAIL rate

Deterministic by default · every value traceable to its source row · nothing invented